

Everything you've typed so far

Every command, meta-command, and function actually used hands-on across this project — grouped by tool, in the order you met them, not alphabetized. Appended at the end of each build, not written once and frozen.

COVERS

Build 1 (agent) & Build 2 (pipeline) — both complete

ENVIRONMENT

Windows 11 · PowerShell · Docker Desktop

DATABASES

pg_local (appdb)

Docker

psql

PowerShell

Git

psycpg

pandas

pandera

pytest

Python stdlib

Docker

CONTAINER ORCHESTRATION

Starting the stack, and getting SQL in and out of a running container.

`docker compose up -d --build`

Build images if anything changed, then start every service in the background.

`docker ps`

List running containers — the first check when a connection unexpectedly fails.

`docker exec -it pg_local psql -U devuser -d appdb`

Open a **live, interactive** psql session inside the Postgres container.

`Get-Content file.sql -Raw | docker exec -i pg_local psql -U devuser -d appdb`

Run a whole .sql file inside the container **non-interactively**, PowerShell-style — < redirection doesn't exist in PowerShell, so the file gets piped in instead. (Bash/cmd would use `docker exec -i ... < file.sql`.)

`docker compose down -v`

Stop everything and delete volumes too — next start re-initializes DB passwords from .env. Used deliberately, not casually.

psql

META-COMMANDS & DDL PATTERNS

Once you're inside a psql session — both the backslash meta-commands and the recurring SQL shapes.

```
\d table_name
```

Describe a table — every column, its type, and any constraints. The one check that settles "did my CREATE TABLE actually work."

```
\du role_name
```

Describe a role — confirms it has no Superuser/Creatorole/Createdb it shouldn't.

```
\pset null 'marker'
```

Display NULLs as a visible marker instead of blank — otherwise a NULL and an empty string look identical.

```
\! cls
```

Shell out and run an OS command without leaving psql — `cls` clears the terminal screen.

```
\q
```

Quit psql, back to the shell prompt.

```
CREATE SCHEMA IF NOT EXISTS name;
```

Create a schema, skip quietly if it's already there — safe to rerun.

```
DROP TABLE IF EXISTS name;
```

Drop a table, skip quietly if it doesn't exist — what makes a rebuild script rerunnable from a blank database.

```
GRANT SELECT ON ALL TABLES IN SCHEMA s TO  
role;
```

Give a role read access to every table **currently** in a schema.

```
ALTER DEFAULT PRIVILEGES IN SCHEMA s GRANT  
SELECT ON TABLES TO role;
```

Extend that same grant automatically to any table created **later** — without this, new tables start private.

```
col TYPE GENERATED ALWAYS AS (expr) STORED
```

A column Postgres computes and locks from other columns in the same row — structurally impossible for it to drift out of sync.

PowerShell

HOST-SIDE SHELL

Running Python, checking your own environment, and vetting/installing system-level tools from outside any container.

```
.\.venv\Scripts\python.exe script.py
```

Run a script with the project's own virtual-env interpreter directly — sidesteps PATH/activation issues entirely.

```
.\.venv\Scripts\python.exe -m pip install pkg
```

Install a package into that specific virtual env, not wherever PATH happens to point.

```
.\.venv\Scripts\python.exe -m pip show pkg
```

Check whether a package is installed, and exactly which version.

```
Get-ChildItem Env: | Where-Object { $_.Name -  
like "*X*" }
```

List environment variables whose name matches a pattern — e.g. catching a stray API key leaked into the shell.

```
Get-Content file
```

Print a file's contents — the go-to way to verify what actually got saved to disk.

```
claude auth status
```

Confirm this Claude Code session is on subscription login, not silently metered API billing.

```
winget show --id pkg -e
```

Pull a package's full manifest — publisher, source URL, license, install hash — before trusting or installing it.

```
winget install --id pkg -e
```

Install a package system-wide once it's been vetted, not before.

Git

CHECKPOINTING PROGRESS

Committing at the end of every phase, per the project's own working style.

`git status`

What's changed, staged, or untracked, right now.

`git add file`

Stage a specific file by name — never a blind `-A`.

`git commit -m "..."`

Record the staged snapshot with a message explaining **why**, not just what.

`git mv old new`

Rename a tracked file while git keeps its history attached to the new name.

`git log --oneline -n`

Recent commit history, one line each — the fastest way to check "where are we, really."

`git diff`

Exact unstaged changes, line by line, before they get staged.

Python — psycopg

TALKING TO POSTGRES

The driver behind every script in this project — agent.py, tools.py, ingest.py, profile.py.

```
psycopg.connect(host=, dbname=, user=,  
password=)
```

Open a connection to a Postgres database.

```
with conn.cursor() as cur:
```

Open a cursor scoped to a with block — closes itself automatically, even on error.

```
cur.execute(sql, params)
```

Run one SQL statement, with parameters passed safely (never string-formatted into the query).

```
cur.executemany(sql, rows)
```

Run the same statement once per row — how ingest.py loads 1,535 rows in one call.

```
cur.fetchall()
```

Pull every result row back as a Python list.

```
cur.description
```

Metadata about the result's columns — used to grab column names generically.

```
conn.commit()
```

Make written changes permanent.

```
conn.close()
```

Release the connection.

Python — pandas

PROFILING RAW DATA

Everything `profile_raw_data.py` used to understand `raw.allocations` before designing the clean schema. (Renamed from `profile.py` in Build 3 — it was silently shadowing Python's `stdlib` `profile` module.)

```
pd.DataFrame(rows, columns=cols)
```

Build a `DataFrame` straight from plain rows + column names — no SQLAlchemy required.

```
df["col"].unique()
```

Every distinct value in a column, once each — printed with `repr()` to expose hidden whitespace.

```
df["col"].value_counts(dropna=False)
```

Count of each distinct value, including blanks/NaN.

```
pd.to_datetime(s, format=, errors="coerce")
```

Parse a column of date strings against an exact format; anything that doesn't match becomes `NaT` instead of crashing the run.

```
series.dt.total_seconds()
```

Convert a time difference into a plain number of seconds.

```
series.str.fullmatch(pattern)
```

Test every value in a column against a regex at once — used to confirm two columns were digit-only.

Python — pandera

DATA-QUALITY SCHEMAS

`test_data_quality.py` uses this to declare what "correct" looks like for `clean.allocations`, as data instead of ad hoc checks.

```
import pandera.pandas as pa
```

The explicit pandas-backend import for modern pandera versions.

```
pa.DataFrameSchema({ ... })
```

Declare an entire `DataFrame`'s expected shape — one entry per column — as data, not scattered asserts.

```
pa.Column(type, checks, nullable=)
```

One column's expected type, whether NULLs are allowed, and any constraints.

```
pa.Check.ge(0)
```

A reusable constraint — here, "greater than or equal to 0" — attached to a `Column`.

```
schema.validate(df)
```

Check a real `DataFrame` against the schema; raises a real error the moment something doesn't match.

pytest

RUNNING THE TEST SUITE

First automated test suite in the project — codifies checks that were previously done by hand.

`.\.venv\Scripts\python.exe -m pytest -v`

Discover and run every `test_*` function in the project, printing each one's name and pass/fail individually.

`def test_something():`

Any function named `test_...` in a `test_*.py` file is auto-discovered — no registration needed.

`assert condition`

Plain Python assert — pytest reports exactly which assertion failed and the actual vs. expected values.

Python — standard library

CSV · OS · DOTENV

No extra install required — these ship with Python itself, or python-dotenv.

`csv.DictReader(f)`

Read a CSV where each row comes back as a `{column: value}` dict — chosen over pandas for ingestion specifically so nothing gets silently type-coerced.

`open(path, newline="", encoding="utf-8-sig")`

Open a file safely for CSV reading, transparently stripping a leading BOM if present.

`os.environ["KEY"]`

Read a required env var — raises immediately if it's missing, rather than continuing with a silent gap.

`os.environ.get("KEY", default)`

Read an optional env var, falling back to a default (e.g. `POSTGRES_HOST` defaulting to `127.0.0.1` outside Docker).

`load_dotenv(encoding="utf-8-sig")`

Load `.env` into the process environment — BOM-safe, matching how Windows tools often save that file.

Living document — appended at the end of each build, not written once and frozen. Reflects hands-on usage through Build 2 (complete). Not exhaustive documentation for any of these tools, just what's actually been typed in this project so far.